CLASE 27

APIs y Requests

Obteniendo datos del mundo exterior

- Que es una API y para que sirve
- HTTP: el idioma de internet
- Libreria requests para consumir APIs
- JSON: el formato universal de datos
- Proyecto: Consultar APIs reales

Que es una API?

API = Application Programming Interface

Analogia del Restaurante

TU (Cliente)

Pides datos

→

MESERO (API)

Intermediario

→

COCINA (Servidor)

Procesa y devuelve

Una API es un CONTRATO: "Si me pides X, te doy Y"

HTTP: El idioma de internet

GET

Obtener datos
(leer)

POST

Enviar datos
(crear)

PUT

Actualizar datos
(modificar)

DELETE

Eliminar datos
(borrar)

Equivalencia con CRUD (bases de datos):

```
GET    =  Read   (SELECT)        POST   =  Create  (INSERT)
PUT    =  Update (UPDATE)       DELETE  =  Delete  (DELETE)
```

Hoy nos enfocamos en GET (obtener datos). POST y los demas los veran en Django.

Status Codes: La respuesta del servidor

Exitosos (2xx)

200 OK - Todo bien, aqui estan los datos

201 Created - Se creo correctamente

Errores del cliente (4xx)

400 Bad Request - Peticion mal formada

401 Unauthorized - Falta autenticacion

404 Not Found - No existe lo que pediste

Errores del servidor (5xx)

500 Internal Error - El servidor fallo

503 Unavailable - Servidor caido

200 = Todo bien | 4xx = Problema tuyo | 5xx = Problema del servidor

requests: La libreria para APIs en Python

"HTTP for Humans" - La libreria mas popular para peticiones HTTP

El patron basico:

1. Importar requests
2. Hacer la peticion (GET, POST, etc.)
3. Verificar el status code
4. Obtener los datos (JSON, texto, etc.)

La respuesta contiene:

`status_code`	El codigo (200, 404, etc.)
`text`	El contenido como string
`json()`	El contenido parseado como diccionario
`headers`	Los headers de la respuesta

GET Request: Obtener datos

El patron mas comun:

requests.get(url) → Hace la peticion GET

response.status_code → Codigo de respuesta

response.json() → Convierte JSON a dict

Por que verificar status_code?

- No siempre funciona
- json() puede fallar
- Siempre validar primero

Practiquemos con JSONPlaceholder: https://jsonplaceholder.typicode.com

JSONPlaceholder - API de prueba perfecta para aprender

```
/posts      →  100 posts falsos
/users      →  10 usuarios con info completa
/comments   →  500 comentarios
/posts/1    →  Un post específico (por ID)
```

Parametros en GET: Filtrar datos

URL con parametros:

```
https://api.example.com/posts?userId=1&limit=5
```

Con requests, usamos el parametro 'params':

```
params = {'userId': 1, 'limit': 5}
requests.get(url, params=params)
```

```
Mal:  url + '?userId=' + str(id)
```

```
Bien:  requests.get(url, params={})
```

Parametros comunes en APIs:

```
Filtros: ?category=tech
Busqueda: ?search=python
Paginacion: ?page=2&limit=10
Ordenamiento: ?sort=date&order=desc
```

JSON: El lenguaje de las APIs

JSON = JavaScript Object Notation | El formato estandar para intercambiar datos

JSON vs Python

JSON	Python
`{ }`	`dict { }`
`[ ]`	`list [ ]`
`"texto"`	`str "texto"`
`123`	`int 123`
`true / false`	`True / False`
`null`	`None`

Son practicamente iguales:

JSON:

```
{"name": "Ana",
 "age": 25,
 "active": true}
```

Python dict:

```
{"name": "Ana",
 "age": 25,
 "active": True}
```

response.json() convierte JSON a diccionario Python automaticamente

Explorando respuestas JSON

Tip 1: pprint para ver bonito

```
from pprint import pprint
pprint(response.json())
```

Muestra el JSON indentado y legible

Tip 2: keys() para explorar

```
data = response.json()
print(data.keys())
```

Ver que campos tiene el diccionario

Tip 3: Verificar tipo de dato

```
data = response.json()
if isinstance(data, list):
```

Saber si es lista o diccionario

Cuando no conoces la API: 1. Peticion simple → 2. pprint → 3. keys() → 4. Listo!

EJERCICIO RAPIDO: JSONPlaceholder

Usando la API: https://jsonplaceholder.typicode.com

1 **Obtener TODOS los usuarios** `GET /users`

Mostrar nombre y email de cada uno

2 **Posts del usuario 3** `GET /posts?userId=3`

Mostrar el titulo de cada post

3 **Comentarios del post 1** `GET /comments?postId=1`

Contar cuantos hay y mostrar los emails

Tiempo: 10 minutos

BREAK - 10 MINUTOS

Volvemos con: APIs reales (GitHub, Clima) + Proyecto final

GitHub API: Datos reales

https://api.github.com - API publica muy completa

Endpoints (sin autenticacion):

```
/users/{username}          →  Info de un usuario
/users/{username}/repos    →  Repositorios
/repos/{owner}/{repo}      →  Info de un repo
```

Ejemplos:

/users/torvalds
Creador de Linux

/repos/python/cpython
Codigo fuente de Python

Sin autenticacion: 60 peticiones por hora (suficiente para practicar)

Consultemos usuarios y repositorios reales en VS Code!

Open-Meteo: API del clima

API de clima gratuita y sin API key | https://open-meteo.com

Endpoint principal:

```
api.open-meteo.com/v1/forecast
  ?latitude=X
  &longitude=Y
  &current_weather=true
```

Datos que devuelve:

`temperature`	→	`Temp actual`
`windspeed`	→	`Velocidad viento`
`weathercode`	→	`Codigo clima`

Coordenadas de ejemplo:

Cd. de Mexico: 19.43, -99.13

Madrid: 40.42, -3.70

Buenos Aires: -34.61, -58.38

Bogota: 4.71, -74.07

Consultemos el clima en vivo!

Otras APIs para explorar

PokeAPI

Datos de Pokemon

`pokeapi.co/api/v2/pokemon/{nombre}`

Random User

Usuarios fake con foto, nombre, email

`randomuser.me/api/`

Dog CEO

Fotos random de perros

`dog.ceo/api/breeds/image/random`

Todas sin API key - Experimenten!

Lista de APIs publicas: github.com/public-apis/public-apis

Manejo de errores con APIs

Errores comunes:

- Timeout: Servidor tarda mucho
- ConnectionError: Sin internet
- 404: Recurso no existe
- 500: Error del servidor
- JSON invalido: Respuesta no es JSON

Patron robusto:

```
try:
  response = requests.get(
    url, timeout=10)
  response.raise_for_status()
  data = response.json()
except Timeout:
  ...
except RequestException:
  ...
```

Conceptos clave:

`timeout=10`	Esperar maximo 10 segundos (evita que el programa se cuelgue)
`raise_for_status()`	Lanza excepcion automatica si status != 2xx
`try/except`	Captura errores de conexion, timeout, HTTP, JSON

Siempre usar timeout para evitar que el programa se cuelgue. Siempre try/except en produccion.

PROYECTO: Consultor de GitHub

```
=== CONSULTOR DE GITHUB ===
1. Buscar usuario
2. Ver repositorios de un usuario
3. Ver detalles de un repositorio
4. Comparar seguidores de 2 usuarios
0. Salir
```

Requisitos:

- Manejo de errores
- Formateo bonito de datos
- Timeout en todas las peticiones
- Funciones separadas por operacion

Endpoints a usar:

```
GET /users/{username}
GET /users/{username}/repos
GET /repos/{owner}/{repo}
```

Tiempo: 20 minutos
Pueden trabajar en parejas

Revision del proyecto

Funcion reutilizable para peticiones	`hacer_peticion() con manejo de errores`
Timeout en todas las peticiones	`requests.get(url, timeout=10)`
Manejo de errores completo	`try/except para conexion y HTTP errors`
Menu interactivo funcionando	`Loop principal con opciones`
Datos formateados bonitos	`Alineacion, informacion clara`
Funciones separadas	`Una funcion por operacion del menu`

Este patron es reutilizable para cualquier API

Resumen - APIs y Requests

GET request	`requests.get(url)`
Con parametros	`requests.get(url, params={})`
Status code	`response.status_code`
Obtener JSON	`response.json()`
Timeout	`requests.get(url, timeout=10)`
Verificar errores	`response.raise_for_status()`

PROXIMA FASE: DJANGO (Clases 28-34)

Proyecto: Dashboard E-commerce

- Landing page publica
- Sistema de login/registro
- Dashboard con metricas y analisis
- CRUD de productos/ventas
- Deploy a produccion

De CONSUMIR APIs → a CREAR sus propias APIs